

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 24
Recap

Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

期末考试说明

- 理论课（闭卷）
  - 日期：**2025年1月15日（周三）**
  - 地点：东校园教学楼 [redacted] 室
  - 时间：上午14:30-16:30（**2 小时**）
  - 题型（全课程，百分制）
    - 单项选择题（20题，40分）
    - 综合应用题（6题，60分）

- 实验课
  - 题型：编程题（全课程，百分制，**共10题**）
  - 日期：**2024年1月16日（周四）**
  - 地点：东校园南实验楼 [redacted] 室
  - 时间：下午14:00-17:00（**3 小时**）
  - 系统：Matrix + Windows，与平时实验课环境相同
  - 要求：**不可带书，不带任何纸质材料**，到时统一发放草稿纸，
    除访问考试网页外，开放离线网站cppreference等在线手册，供考试时参考，
    提交次数限制最多 **10** 次。

- 理论课和实验课的计分比例：
  - 理论课：平时40% + 期末60%
  - 实验课：平时70% + 期末30%（需要**完整**做出**2**道题目，机考通过少于2题，总评成绩不及格，必须**补考**或**重修**）
- 实验课的平时成绩，只有在上机考试做出2题的基础上，才有计分意义。
- 做出多于2题的，期末考试**以60分起计，每多一题增加5分**。

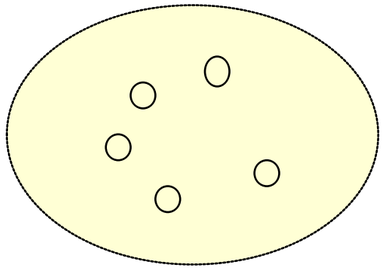
用 stl、cppreference

计算机学院2024学年第一学期本科课程考试安排表

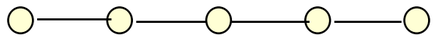
月	日	星期	考试时间	年级、专业（班别）	课程类别	课程号	考试科目	人数	教学班号	考试地点	考试方式	监考组长	监考组员	巡考	备注
1	15	三	14：30-16：30	23级计科（系统结构）+23级信计、23辅修、	专必	DCS211	数据结构与算法	85		东校区B205	闭卷	刘聪	胡凤娟、丘南海	刘宁	
1	15	三	14：30-16：30	23级计科1班	专必	DCS211	数据结构与算法	100		东校区B201	闭卷	苏卓	吴岚岚、李湄、吴晓鸿、侯艳冰	刘宁	
1	15	三	14：30-16：30	23级计科2班	专必	DCS211	数据结构与算法	97		东校区B203	闭卷	张子臻	成振芳、梁穗、张惠	刘宁	
1	15	三	14：30-16：30	23级计科3班	专必	DCS211	数据结构与算法	96		东校区B204	闭卷	魏朋旭、林倞	陈思亦、魏东、甘德秀	刘宁	
1	15	三	14：30-16：30	23级计科（人工智能）+23级保密、23级辅修	专必	DCS211	数据结构与算法	67		东校区B202	闭卷	蔡穗华	廖雪莹、杨玉健	刘宁	
1	16	四	14：00-17：00	23级计科1班	专必	DCS213	数据结构与算法实验	100		东校实验中心D503&D402	闭卷	苏卓	胡凤娟、阮琪	万海	
1	16	四	14：00-17：00	23级计科2班	专必	DCS213	数据结构与算法实验	100		东校实验中心D501	闭卷	张子臻	吴岚岚	万海	
1	16	四	14：00-17：00	23级计科3班	专必	DCS213	数据结构与算法实验	96		东校实验中心北楼B203	闭卷	魏朋旭、林倞	李湄	万海	
1	16	四	14：00-17：00	23级计科（人工智能）+23级保密、23级辅修、辅微、辅修学士学位	专必	DCS213	数据结构与算法实验	69		东校实验中心D403	闭卷	蔡穗华	成振芳	万海	
1	16	四	14：00-17：00	23级计科（系统结构）+23级信计、23辅修、辅微、辅修学士学位	专必	DCS213	数据结构与算法实验	83		东校实验中心E403	闭卷	刘聪	丘南海	万海	

Classification of data structures

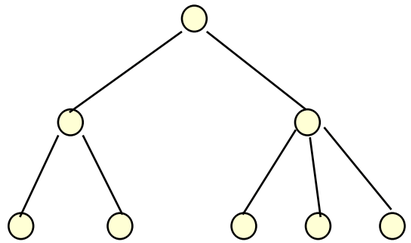
- 数据结构是相互间具有某些关系的数据元素的集合。它包括下面三方面的内容：
  1）**数据的逻辑结构**：表示数据元素间的关系；
  2）**数据的存储结构**：指如何在计算机上存储数据元素及其关系；
  3）**数据的运算（集合）**：是数据元素集上所允许的操作



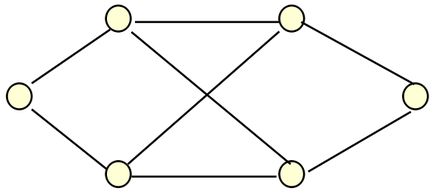
(a) set



(b) linear



(c) tree



(d) graph

List of data structures

- Data type
  - Primitive types: Boolean, Integer, Double, Character, Enum, etc.
  - Composite types: Array, Struct, Union, etc.
  - Abstract data types: Stack, List, Map, Set, String, Queue, etc.
- Linear data structures
  - Arrays
  - Lists
- Trees
  - Binary trees
  - Heap
  - Multiway trees
  - …
- Graphs

Abstract Data Type (ADT)

- An Abstract Data Type (ADT) is a data type that is organized in such a way that the specification on the objects and specification of the operations on the objects are separated from the representation of the objects and the implementation on the operations.
- 用三元组描述如下：

  ADT = （D,S,P）

其中：D是数据对象，S是D上的关系集，P是D的基本操作集。

ADT 抽象数据类型名{

  数据对象：〈数据对象的定义〉

  数据关系：〈数据关系的定义〉

  基本操作：〈基本操作的定义〉

} ADT 抽象数据类型名

Complexity

- The complexity of an algorithm is a “device-independent” measure of how much time it consumes. Rather than expressing the time consumed in seconds, we attempt to count how many “elementary operations” the algorithm performs when presented with instances of different sizes.
- Asymptotic notations: Θ, O, Ω, o, ω.
  - Big-O notation: It defines an asymptotic upper bound for a function *f(n)*.
  - **Definition:** A function *f(n)* is said to be *O(g(n))* if there are constants *c* and n_0 such that $f(n) \leq cg(n), \ \forall \ n \geq n_0$.
- Order of Growth
  - The rate of growth, or the order of growth, possesses the highest significance.
  - Typical order of growth: O(1), O(log *n*), O(√*n*), O(*n*), O(*n*log *n*), O(n^2), O(n^3), O(2^n), O(*n*!).

Stacks

- A stack is a data structure in which all insertions and deletions of entries are made at one end, called the top of the stack. The last entry which is inserted is the first one that will be removed. The operations perform in a Last-In-First-Out (LIFO) manner.
- Implementing a Contiguous Stack
  - We set up an array to hold the entries in the stack and a counter to indicate how many entries there are.
- Implementing a Linked Stack
  - A linked structure is made up of nodes, each containing both the information that is to be stored as an entry of the structure and a pointer telling where to find the next node in the structure.



Class Declaration for Stacks

```
const int maxstack = 10;

class Stack {
  public:
  Stack( );
  bool empty( ) const;
  Error_code pop( );
  Error_code top(Stack entry &item) const;
  Error_code push(const Stack_entry &item);
private:
  int count;
  Stack_entry entry[maxstack];
};
```

Contiguous Stack Implementation

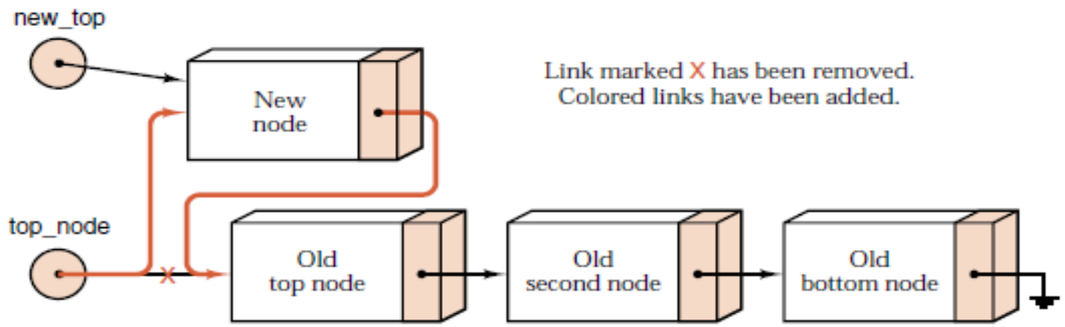
```
class Stack {
public:
  Stack( );
  bool empty( ) const;
  Error_code push(const Stack_entry &item);
  Error_code pop( );
  Error_code top(Stack_entry &item) const;
  // Safety features for linked structures
  ~Stack( );
  Stack(const Stack &original);
  void operator = (const Stack &original);
protected:
  Node *top_node;
};
```

Linked Stack Implementation

Linked Stack Operations

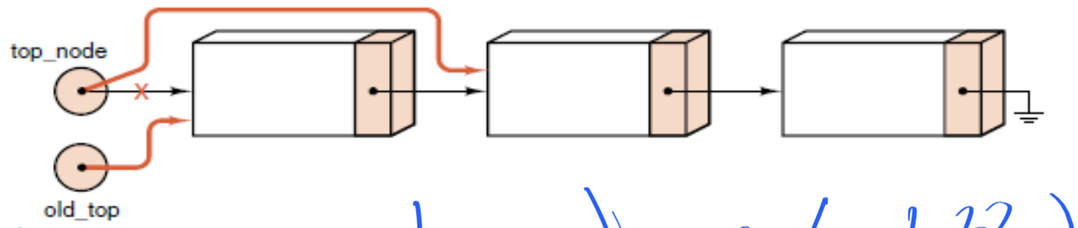
Pushing a Linked Stack

```
Error_code Stack :: push(const Stack_entry &item)
/* Post: Stack entry item is added to the top of the Stack;
returns success or returns a code of overflow if dynamic
memory is exhausted. */
{
        Node *new_top = new Node(item, top_node);
        if (new_top == NULL) return overflow;
        top_node = new_top;
        return success;
}
```



Popping a Linked Stack

```
Error_code Stack :: pop( )
/* Post: The top of the Stack is removed. If the
Stack is empty the method returns underflow;
otherwise it returns success. */
{
        Node *old_top = top_node;
        if (top_node == NULL) return underflow;
        top_node = old_top->next;
        delete old_top;
        return success;
}
```



Queues

- **Definition**: A queue is a list in which all additions to the list are made at one end, and all deletions from the list are made at the other end. Queues are also called ***first-in, first-out lists,*** or ***FIFO*** for short.
- The first entry that will be removed from the queue, is called the *front* of the queue (or, sometimes, the *head* of the queue).
- The last entry in the queue, that is, the one most recently added, is called the *rear* (or the *tail*) of the queue.

Class Declaration for Queues

```
const int maxqueue = 10;

class Queue {
public:
  Queue( );
  bool empty( ) const;
  Error_code serve( );
  Error_code append(const Queue_entry
&item);
  Error_code retrieve(Queue_entry &item)
const;
protected:
  int count;
  int front, rear;
  Queue_entry entry[maxqueue];
};
```

Contiguous Stack Implementation

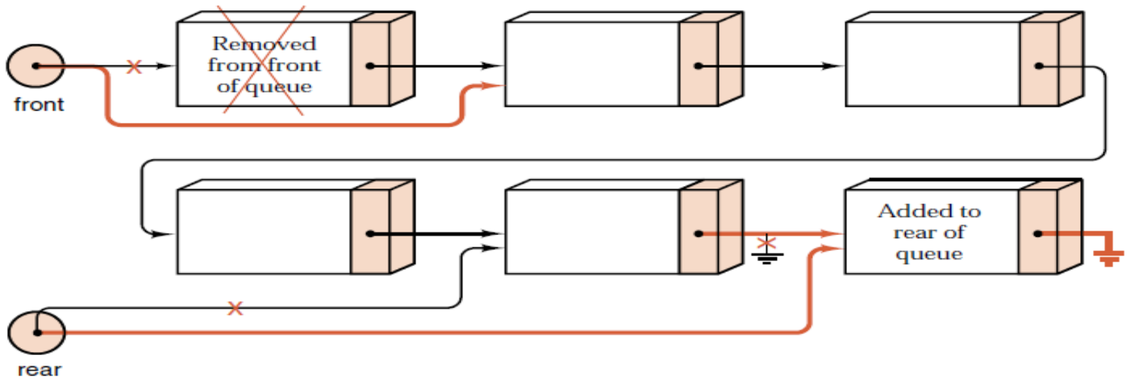
```
class Queue {
public:
  // standard Queue methods
  Queue( );
  bool empty( ) const;
  Error_code append(const Queue_entry &item);
  Error_code serve( );
  Error_code retrieve(Queue entry &item) const;
  // safety features for linked structures
  ~Queue( );
  Queue(const Queue &original);
  void operator = (const Queue &original);
protected:
  Node *front, *rear;
};
```

Linked Stack Implementation

Linked Queue Methods

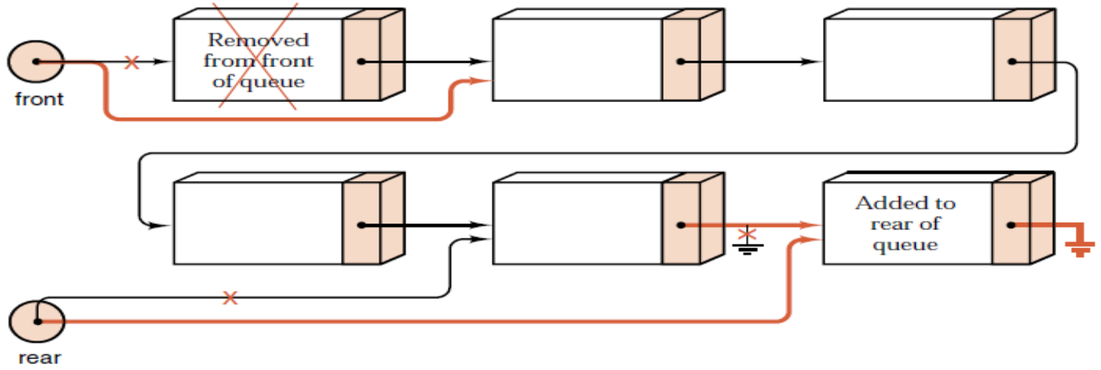
Append an entry:

```
Error_code Queue :: append(const Queue_entry &item)
/* Post: Add item to the rear of the Queue and return a code of
success or return a code of overflow if dynamic memory is
exhausted. */
{
        Node *new_rear = new Node(item);
        if (new_rear == NULL) return overflow;
        if (rear == NULL) front = rear = new_rear;
        else {
                rear->next = new_rear;
                rear = new_rear;
        }
        return success;
}
```



Serve an entry:

```
Error_code Queue :: serve( )
/* Post: The front of the Queue is removed. If the Queue is empty,
return an Error code of underflow. */
{
        if (front == NULL) return underflow;
        Node *old_front = front;
        front = old_front->next;
        if (front == NULL) rear = NULL;
        delete old_front;
        return success;
}
```



Circular implementation of queues

```
Queue :: Queue( )
/* Post: The Queue is initialized to be empty. */
{
    count = 0;
    rear = maxqueue - 1;
    front = 0;
}
Error_code Queue :: append(const Queue_entry &item)
/* Post: item is added to the rear of the Queue. If the Queue is full return
an Error_code of overflow and leave the Queue unchanged. */
{
    if (count >= maxqueue) return overflow;
    count++;
    rear = ((rear + 1) == maxqueue) ? 0 : (rear + 1);
    entry[rear] = item;
    return success;
}
Error_code Queue :: serve( )
/* Post: The front of the Queue is removed. If the Queue is empty return an
Error code of underflow. */
{
    if (count <= 0) return underflow;
    count--;
    front = ((front + 1) == maxqueue) ? 0 : (front + 1);
    return success;
}
```

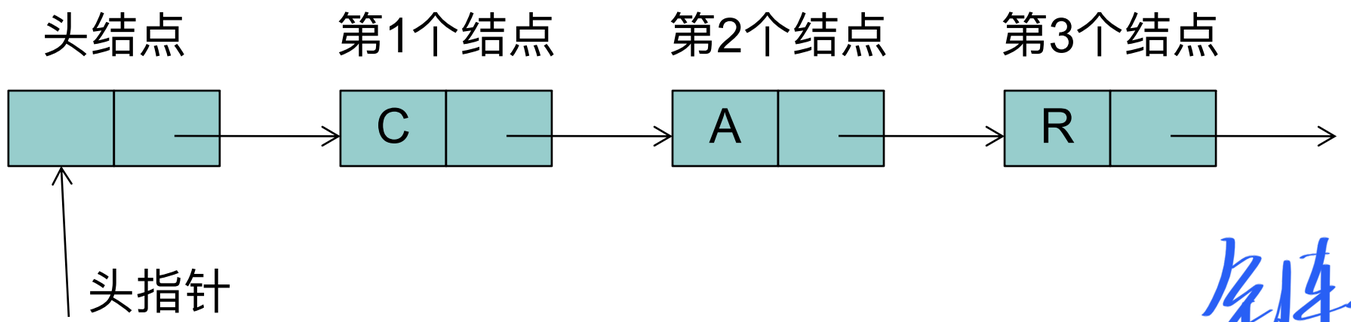
Note boundary cases

Lists and Strings

- A **list** of elements of type T is a finite sequence of elements of T together with the following operations:
  1. **Construct** the list, leaving it empty.
  2. Determine whether the list is **empty** or not.
  3. Determine whether the list is **full** or not.
  4. Find the **size** of the list.
  5. **Clear** the list to make it empty.
  6. **Insert** an entry at a specified position of the list.
  7. **Remove** an entry from a specified position in the list.
  8. **Retrieve** the entry from a specified position in the list.
  9. **Replace** the entry at a specified position in the list.
  10. **Traverse** the list, performing a given operation on each entry.

Head Pointer and Head Node

- 链表的第一个结点的存储位置叫做头指针。
- 有时候为了更方便对链表进行操作，会在单链表的第一个结点前附设一个结点，称为头结点。头结点的数据域可以不存储任何信息，也可以存储线性表的长度等附加信息。头结点的指针域存储指向第一个结点的指针。



链表的操作

Searching

- Sequential search
  - ? The method is simple;
  - ? Efficient for short list;
  - ? The list don't have to be ordered;
  - ✗ Disaster for long list to be searched;
- Binary search
  - In searching an ordered list, first compare the target to the key in the center of the list.
  - If it is smaller, restrict the search to the left half; otherwise restrict the search to the right half, and repeat.
  - In this way, at each step we reduce the length of the list to be searched by half.

O(log n) 最快了！

Binary Search Version 1

```
Error_code binary_search_1 (const Ordered_list &the_list,
        const Key &target, int &position)
/* Post: If a Record in the list has Key equal to target, then position
locates one such entry and a code of success is returned. Otherwise,
not present is returned and position is undefined.
Uses: Methods for classes List and Record. */
{
    Record data;
    int bottom = 0, top = the_list.size( ) - 1;
    while (bottom < top) {
        int mid = (bottom + top) / 2;
        the_list.retrieve(mid, data);
        if (data < target)
            bottom = mid + 1;
        else top = mid;
    }
    if (top < bottom) return not_present;
    else {
        position = bottom;
        the_list.retrieve(bottom, data);
        if (data == target) return success;
        else return not_present;
    }
}
```

When the middle part of the list is reduced to size 1, it will be guaranteed to be the *first* occurrence of the target if it appears more than once in the list.

Binary Search Version 2

```
Error_code binary_search_2(const Ordered_list &the_list,
    const Key &target, int &position)
/* Post: If a Record in the list has key equal to target, then position
locates one such entry and a code of success is returned. Otherwise,
not present is returned and position is undefined.
Uses: Methods for classes Ordered_list and Record. */
{
    Record data;
    int bottom = 0, top = the_list.size( ) - 1;
    while (bottom <= top) {
        position = (bottom + top) / 2;
        the_list.retrieve(position, data);
        if (data == target) return success;
        if (data < target) bottom = position + 1;
        else top = position - 1;
    }
    return not_present;
}
```

Sorting

Name	Average	Worst	Memory	Stable
Bubble sort	n^2	n^2	*1*	Yes
Selection sort	n^2	n^2	*1*	Depends
Shell sort		$n^{1.25}$	*1*	No
Insert sort	n^2	n^2	*1*	Yes
Heapsort	*nlogn*	*nlogn*	*1*	No
Merge sort	*nlogn*	*nlogn*	*n*	Yes
Quick sort	*nlogn*	n^2	*logn*	Depends

Sorting

- Shell sort
  - If we compare entries far apart, then it could sort the entries far apart.
  - Group entries according to increment and then sort entries inside the group.
  - Decrease increment.
  - Finally, the increment between keys being compared is reduced to 1.
- Merge sort
  - If the size of the list is greater than 1, chop(拆分) the list into two sublists of equal sizes.
  - Sort two sublists separately.
  - Merge the two sorted sublists into a single sorted list.
- Quick sort
  - We first choose some key from the list for which, we hope, about half the keys will come before and half after. Call this key the ***pivot***.
  - Then we partition the items so that all those with keys less than the pivot come in one sublist, and all those with greater keys come in another.
  - Then we sort the two reduced lists separately, put the sublists together, and the whole list will be in order.

Heapsort

DEFINITION A ***heap*** is a list in which each entry contains a key, and, for all positions k in the list, the key at position k is at least as large as the keys in positions $2k$ and $2k + 1$, provided these positions exist in the list.

- A binary heap is a complete binary tree.
- The first entry must have the largest/smallest key in the max-heap/min-heap.
- Heapsort proceeds in two phases:
  - Arrange the entries in the list into a heap; (建堆)
  - Repeatedly remove the top of the heap and promote another entry to take its place (restore the heap). (调整堆)

Hashing

- Idea of Hashing:
  - A table can be indexed by a key.
  - If there is a function $f: K \rightarrow N$, where K is the set of possible keys and N is the set of indices of an array, then we can store an entry with key K at the location $f(K)$ and hence access a table entry becomes easy.
- To use hashing we must
  - (a) find good hash functions
  - (b) determine how to resolve collisions.
- Hashing methods
  - Modular arithmetic
  - Folding
  - …

Hashing

- To deal with collisions, there are generally two solutions:

  (1) Open addressing（开发地址）

  - Linear Probing (线性探查): it starts with the hash address and searches sequentially for the target key or an empty position.
  - Quadratic probing (二次探查): if there is a collision at hash address *h*, probes the table at locations $h + 1$, $h + 2^2$, $h + 3^2$, …, that is, at locations $h + i^2$ (mod hashsize) for $i = 1,2,\ldots$

  (2) Chaining （链地址法或拉链法）: When there is a collision, the synonym (同义) is put into a linked list of synonyms.

- The load factor of a table is $\lambda = n/t$, n is the number of entries in the table and t the size of the array.
- λ can be seen as the average length of the chains.

Binary Trees

DEFINITION A ***binary tree*** is either empty, or it consists of a node called the ***root*** together with two binary trees called the ***left subtree*** and the ***right subtree*** of the root.

- Traversal of Binary Trees
  - Inorder
  - Preorder
  - Postorder
  - Level-by-level

Binary Search Trees

A ***binary search tree*** is a binary tree that is either empty or in which every node has a key (within its data entry) and satisfies the following conditions:

1. The key of the root (if it exists) is greater than the key in any node in the left subtree of the root.
2. The key of the root (if it exists) is less than the key in any node in the right subtree of the root.
3. The left and right subtrees of the root are again binary search trees.

- Tree search
- Tree sort
- Tree node insertion
- Tree node removal

AVL Trees

- An *AVL* tree is a binary search tree in which the heights of the left and right subtrees of the root differ by at most 1 and in which the left and right subtrees are again AVL trees.
- With each node of an AVL tree is associated a *balance factor*（平衡因子，左子树的高度减去右子树的高度）that is *left higher*, *equal*, or *right higher* according, respectively, as the left subtree has height greater than, equal to, or less than that of the right subtree.
- Rotation
  - RR
  - LL
  - RL
  - LR

General Trees and Forests

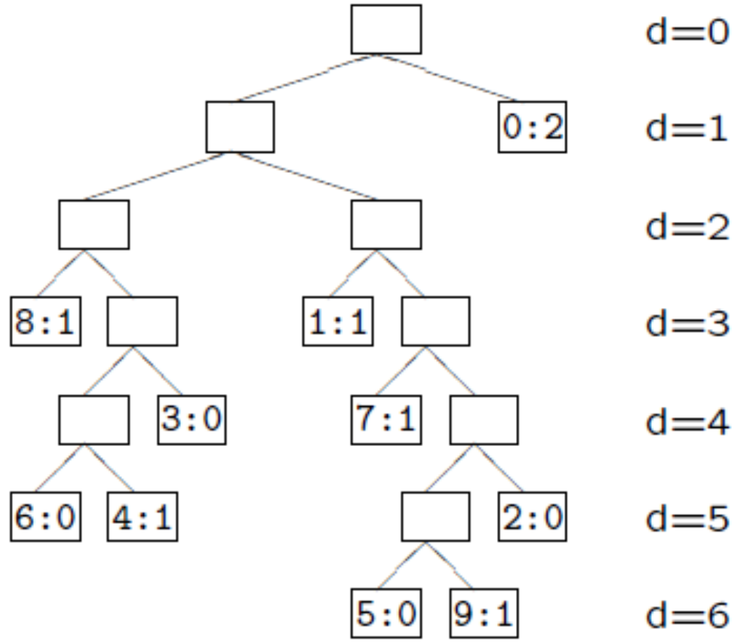
- Tree Representation
  - Parent method
  - Multiple links
  - Child-link
  - First child next sibling
- Forest Traverse
- Disjoint set
  - MAKE-SETs requires O(V).
  - FIND-SETs and UNIONs perform O(E) time.
  - Assuming the implementation of disjoint-set data structure that uses union by rank and path compression, so amortized in O(E α(V)).

Huffman Tree

- Assign to each leaf of the tree a value, *f(c)*, which is the frequency of occurrence of the character *c* represented by the leaf.
- Let $d_T(c)$ be the depth of character *c*'s leaf in the tree *T*.
- Then the number of bits required to encode a file is

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

- which we define as the cost or WPL(Weighted Path Length) of the tree T.



Graphs

- Graph Representation
  - Adjacency Matrix (邻接矩阵)
  - Adjacency Lists (邻接表)
- Graph Traversal
  - Depth-first: It is roughly analogous to preorder traversal of an ordered tree.
  - Breadth-first: It is roughly analogous to level-by-level traversal of an ordered tree. It can be done by using a queue. The head is the vertex to be visited. When it is visited, all its adjacent vertices are put in the queue.

Graphs

- Topological Sorting
  - Let *G* be a directed graph with no cycles (DAG). A topological order for *G* is a sequential listing of all the vertices in *G* such that, for all vertices $v, w \in G$, if there is an edge from *v* to *w*, then *v* precedes *w* in the sequential listing.
- Topological Sorting Algorithms
  - Bread-first algorithm
  - Depth-first algorithm

Graphs

- Minimum Spanning Trees

> DEFINITION A ***minimal spanning tree*** of a connected network is a spanning tree such that the sum of the weights of its edges is as small as possible.

  - Prim's algorithm

```
for (int i = 1; i < count; i++) {
   Vertex v;                          //  Add one vertex v to X on each pass.
   Weight min = infinity;
   for (w = 0; w < count; w++) if (!component[w])
      if (distance[w] < min) {
         v = w;
         min = distance[w]; }
   if (min < infinity) {
      component[v] = true;
      tree.add_edge(v, neighbor[v], distance[v]);
      for (w = 0; w < count; w++) if (!component[w])
         if (adjacency[v][w] < distance[w]) {
            distance[w] = adjacency[v][w];
            neighbor[w] = v;                    }
   }
   else break;                        //  finished a component in disconnected graph
}
```

  - Kruskal's algorithm (union-find algorithm)

Graphs

- Shortest Path
  - Dijkstra's algorithm (single source)

```
found[source] = true;        //  Initialize with vertex source alone in the set S.
distance[source] = 0;
for (int i = 0; i < count; i++) {  //  Add one vertex v to S on each pass.
  Weight min = infinity;
  for (w = 0; w < count; w++) if (!found[w])
    if (distance[w] < min) {
      v = w;
      min = distance[w];
    }
  found[v] = true;
  for (w = 0; w < count; w++) if (!found[w])
    if (min + adjacency[v][w] < distance[w])
      distance[w] = min + adjacency[v][w];
 }
}
```

  - Bellman-ford (single source, detect negative-weight cycle)
  - Floyd-Warshall (all-pairs)

```
for k ← 1 to n do
  for i ← 1 to n do
    for j ← 1 to n do
      if (D_{i,k} + D_{k,j} < D_{i,j}) then
        D_{i,j} ← D_{i,k} + D_{k,j};
```

找队列、编号为6
个人表→考试名称

手动：数据输入：读到文件结束/一行行读入